



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Julia Khrabatyn
03/01/2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies

- ✓ Data collection
- ✓ Data Wrangling
- ✓ EDA with Data Visualization
- ✓ EDA with using SQL
- ✓ Creating an interactive map using Folium
- ✓ Creating an interactive Dashboard using Plotly Dash
- ✓ Creating models

- Summary of all results

- ✓ EDA results
- ✓ Interactive analytics
- ✓ Predictive models

Introduction

- Project background and context

SpaceX advertises Falcon 9 rocket launches on its website with a cost of 62 million dollars; other providers cost upward of 165 million dollars each, much of the savings is because SpaceX can reuse the first stage.

- Problems you want to find answers

The key problem of that project is to predict if the first stage of Falcon 9 rocket will successfully land.



Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
 - By using SpaceX REST API and web scraping
- Perform data wrangling
 - By cleaning and preparing data for next steps (handling missing values, applying one hot encoding)
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - Create, tune and evaluate Logistic Regression Model, K-nearest neighbors Model, Decision Tree Model and Support Vector Machine Model.



Data Collection

- Steps for Data Collection:
 - 1) Importing required libraries
 - 2) Requesting rocket launch data from SpaceX API with the following URL:
<https://api.spacexdata.com/v4/launches/past>
 - 3) Decode the response content as a Json using `.json()` and turn it into a Pandas dataframe using `.json_normalize()`
 - 4) Create dictionary from data
 - 5) Create data frame from dictionary
 - 6) Filter the dataframe to only include Falcon 9 launches



Data Collection – SpaceX API (Main Steps)

1

Requesting rocket launch data from SpaceX API with the following URL: <https://api.spacexdata.com/v4/launches/past>



```
In [6]: spacex_url="https://api.spacexdata.com/v4/launches/past"

In [7]: response = requests.get(spacex_url)
```

2

Decode the response content as a Json using `.json()` and turn it into a Pandas dataframe using `.json_normalize()`



```
In [10]: response=requests.get(static_json_url)

In [11]: response.status_code

Out[11]: 200
```

```
In [23]: # Use json_normalize meethod to convert the json result into a dataframe
data = pd.json_normalize(response.json())
```

3

Use custom functions for data cleaning and preparing for the next steps



```
In [30]: # Call getLaunchSite
getLaunchSite(data)
```

```
In [31]: # Call getPayloadData
getPayloadData(data)
```

```
In [32]: # Call getCoreData
getCoreData(data)
```

Link to the notebook here:



https://github.com/julia-khrabatyn/Data-Science-Project/blob/main/0_jupyter-labs-spacex-data-collection-api.ipynb

Data Collection – SpaceX API (Main Steps)

4

Create dictionary from data



```
In [33]: launch_dict = {'FlightNumber': list(data['flight_number']),  
                        'Date': list(data['date']),  
                        'BoosterVersion': BoosterVersion,  
                        'PayloadMass': PayloadMass,  
                        'Orbit': Orbit,  
                        'LaunchSite': LaunchSite,  
                        'Outcome': Outcome,  
                        'Flights': Flights,  
                        'GridFins': GridFins,  
                        'Reused': Reused,  
                        'Legs': Legs,  
                        'LandingPad': LandingPad,  
                        'Block': Block,  
                        'ReusedCount': ReusedCount,  
                        'Serial': Serial,  
                        'Longitude': Longitude,  
                        'Latitude': Latitude}
```

5

Create data frame from dictionary



```
In [34]: # Create a data from launch_dict  
df = pd.DataFrame(launch_dict)
```

6

Filter the dataframe to only include Falcon 9 launches



```
In [40]: # Hint data['BoosterVersion']!='Falcon 1'  
data_falcon9 = df[df.BoosterVersion == 'Falcon 9']  
data_falcon9
```

Link to the notebook here:



https://github.com/julia-khrabatyn/Data-Science-Project/blob/main/0_jupyter-labs-spacex-data-collection-api.ipynb

Data Collection - Scraping

1

Perform an HTTP GET method to request the Falcon9 Launch HTML page, as an HTTP response



In [22]:

```
# use requests.get() method with the provided static_url and headers
# assign the response to a object
response = requests.get(url=static_url, headers=headers)
```

2

Create a BeautifulSoup object from the HTML response



In [23]:

```
# Use BeautifulSoup() to create a BeautifulSoup object from a response text content
soup = BeautifulSoup(response.content, "html.parser")
```

3

Extract all tables from the HTML table header



In [28]:

```
# Use the find_all function in the BeautifulSoup object, with element type `table`
# Assign the result to a list called `html_tables`
html_tables = list(soup.find_all('table'))
```

4

Extract column names



In [40]:

```
column_names = []

# Apply find_all() function with `th` element on first_launch_table
x = first_launch_table.find_all('th')
# Iterate each th element and apply the provided extract_column_from_header() to get a column name
# Append the Non-empty column name (`if name is not None and len(name) > 0`) into a list called column_names
for element in x:
    if extract_column_from_header(element) is not None and len(extract_column_from_header(element)) > 0:
        column_names.append(extract_column_from_header(element))
```

Check the extracted column names

In [41]:

```
print(column_names)
```

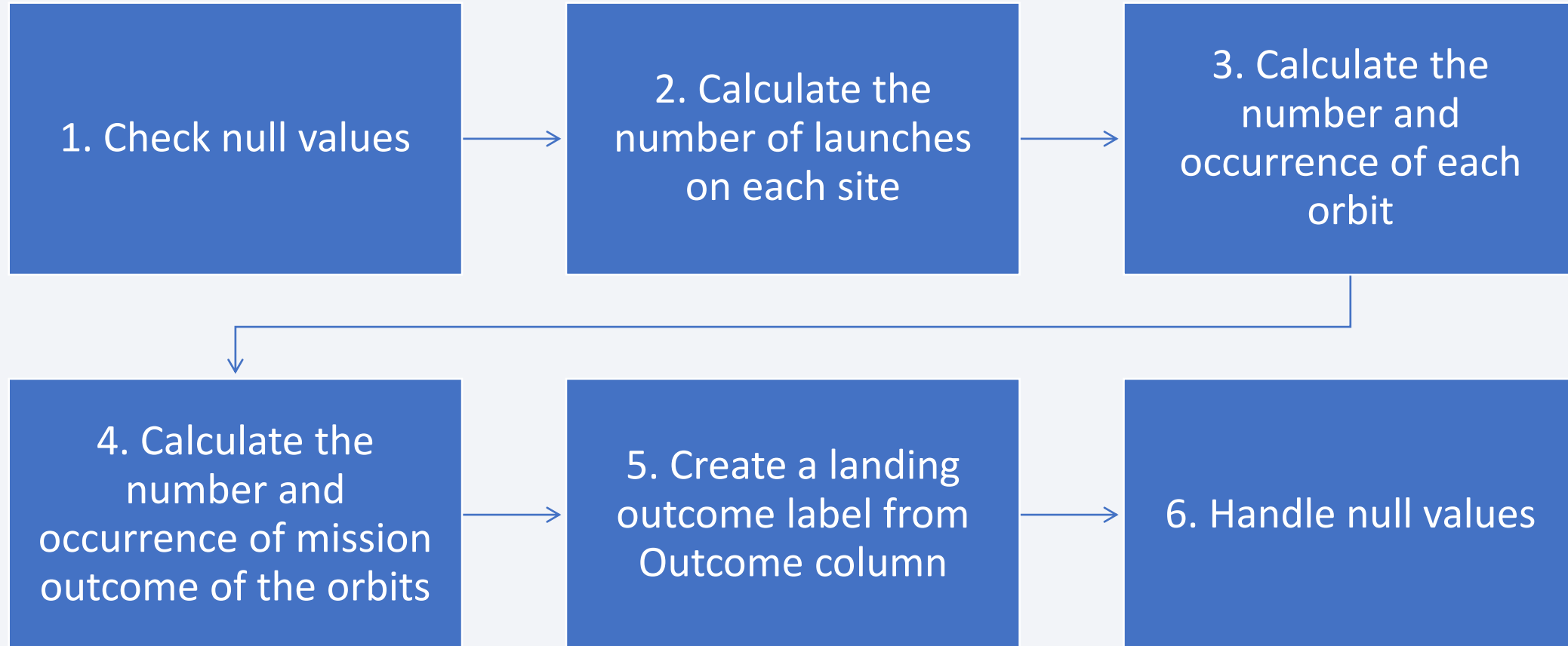
```
['Flight No.', 'Date and time ( )', 'Launch site', 'Payload', 'Payload mass', 'Orbit', 'Customer', 'Launch outcome']
```

Link to the notebook here:



https://github.com/julia-khrabatyn/Data-Science-Project/blob/main/1_jupyter-labs-webscraping.ipynb

Data Wrangling



Link to the notebook here:



https://github.com/julia-khrabatyn/Data-Science-Project/blob/main/2_labs-jupyter-spacex-Data%20wrangling.ipynb

Data Collection - Scraping

5

Fill in the parsed launch record values into dictionary



In [42]:

```
launch_dict= dict.fromkeys(column_names)

# Remove an irrelevant column
del launch_dict['Date and time ( )']

# Let's initial the launch_dict with each value to be an empty list
launch_dict['Flight No.'] = []
launch_dict['Launch site'] = []
launch_dict['Payload'] = []
launch_dict['Payload mass'] = []
launch_dict['Orbit'] = []
launch_dict['Customer'] = []
launch_dict['Launch outcome'] = []
# Added some new columns
launch_dict['Version Booster']=[]
launch_dict['Booster landing']=[]
launch_dict['Date']=[]
launch_dict['Time']=[]
```

6

Create a data frame from dictionary



```
df= pd.DataFrame({ key:pd.Series(value) for key, value in launch_dict.items() })
```

7

Export data to a CSV



```
df.to_csv('spacex_web_scraped.csv', index=False)
```

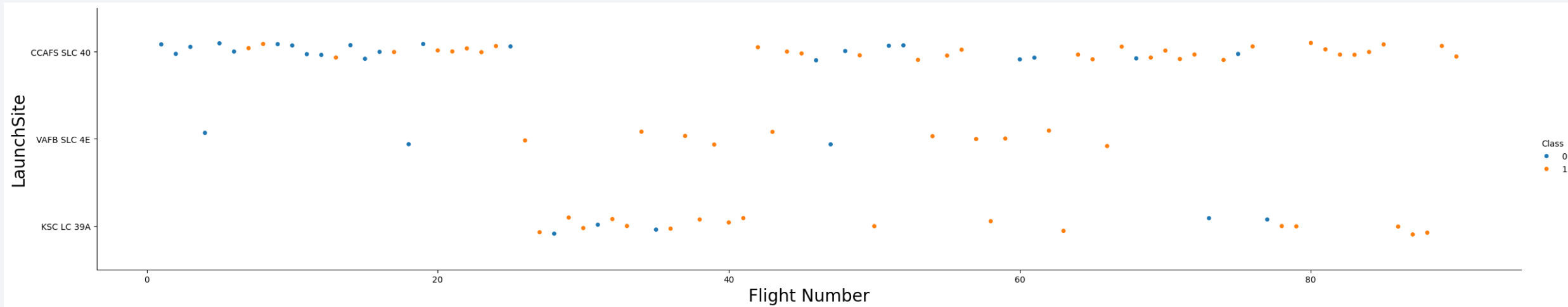
Link to the notebook here:



https://github.com/julia-khrabatyn/Data-Science-Project/blob/main/1_jupyter-labs-webscraping.ipynb

EDA with Data Visualization

- Visualize the relationship between Flight Number and Launch Site - scatter point chart



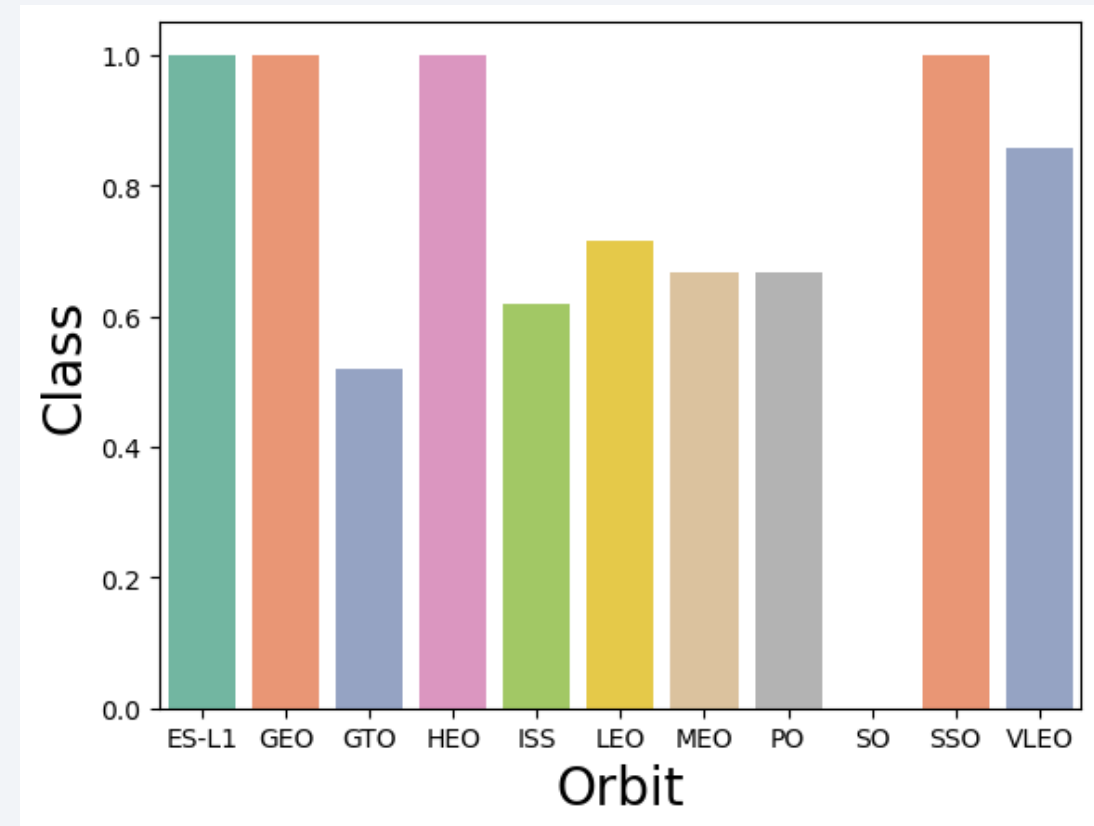
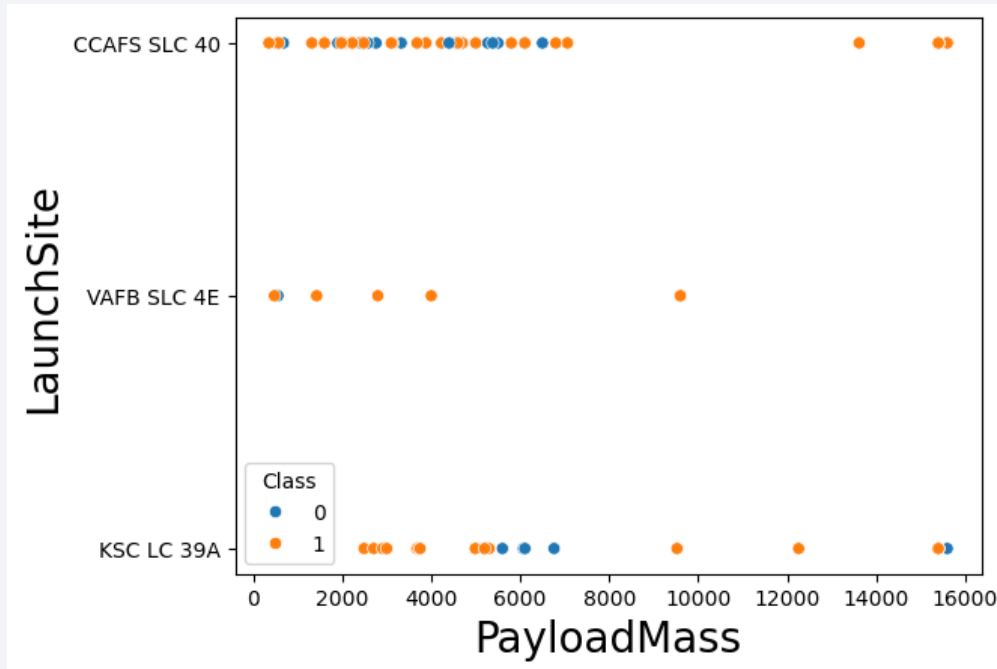
Link to the notebook here:



https://github.com/julia-khrabatyn/Data-Science-Project/blob/main/4_jupyter-labs-data-visualization.ipynb

EDA with Data Visualization

- Visualize the relationship between Payload Mass and Launch Site - scatter point chart
- Visualize the relationship between success rate of each orbit type – bar chart



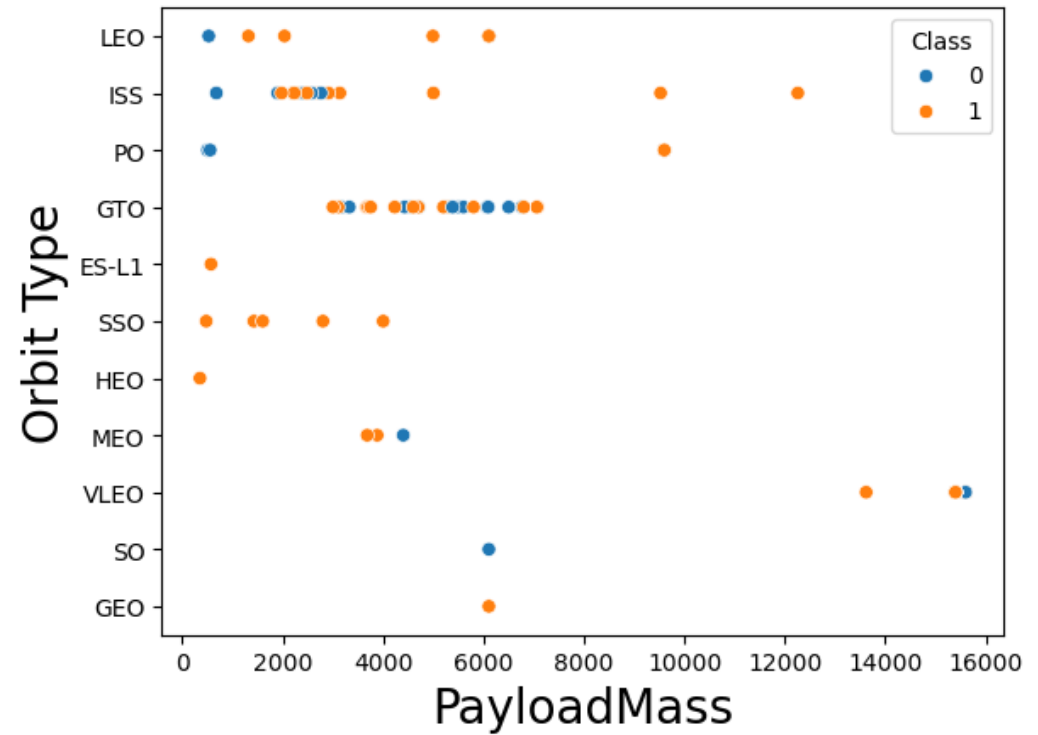
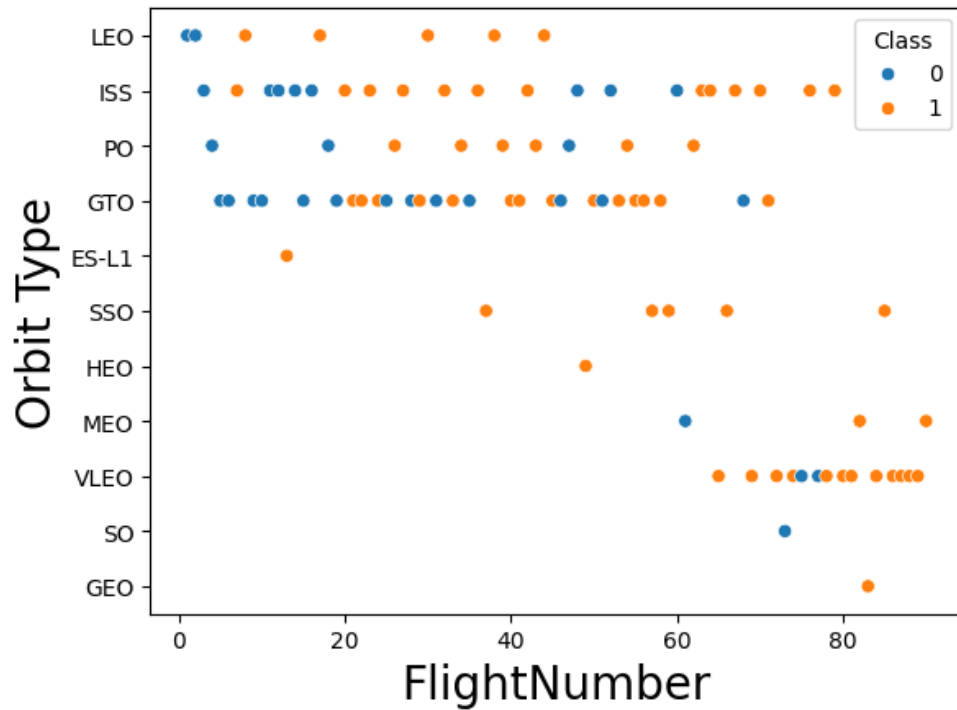
Link to the notebook here:



https://github.com/julia-khrabatyn/Data-Science-Project/blob/main/4_jupyter-labs-data-visualization.ipynb

EDA with Data Visualization

- Visualize the relationship between FlightNumber and Orbit type - scatter point chart
- Visualize the relationship between Payload Mass and Orbit type– scatter point chart



Link to the notebook here:

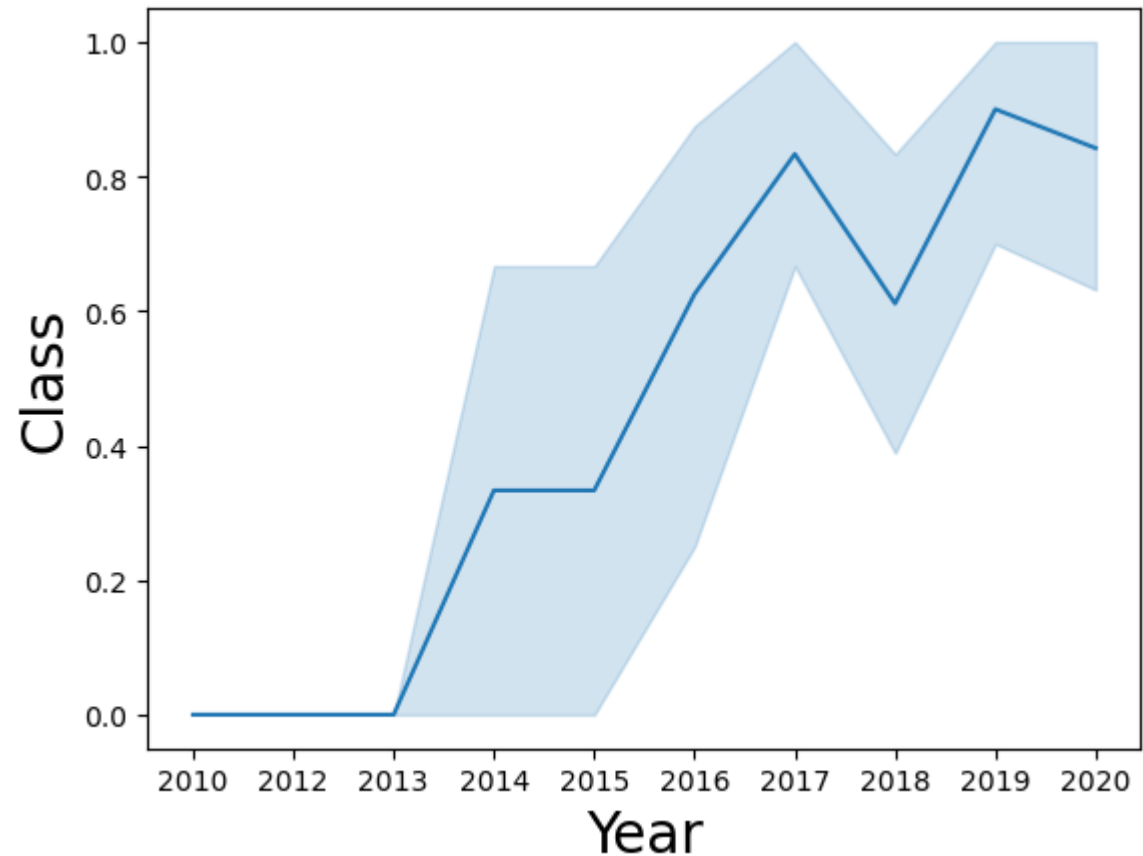


https://github.com/julia-khrabatyn/Data-Science-Project/blob/main/4_jupyter-labs-data-visualization.ipynb

EDA with Data Visualization

- Visualize the launch success yearly trend – line chart

Here we can observe that the success rate since 2013 kept increasing till 2020



Link to the notebook here:



https://github.com/julia-khrabatyn/Data-Science-Project/blob/main/4_jupyter-labs-data-visualization.ipynb

EDA with SQL

- SQL queries was performed to get insight about data and it was used for extracting (for instance):
 - ✓ the names of the unique launch sites in the space mission
 - ✓ the total payload mass carried by boosters launched by NASA (CRS)
 - ✓ average payload mass carried by booster version F9 v1.1
 - ✓ the total number of successful and failure mission outcomes
 - ✓ all the booster_versions that have carried the maximum payload mass

Link to the notebook here:



https://github.com/julia-khrabatyn/Data-Science-Project/blob/main/3_jupyter-labs-eda-sql-coursera_sqlite.ipynb

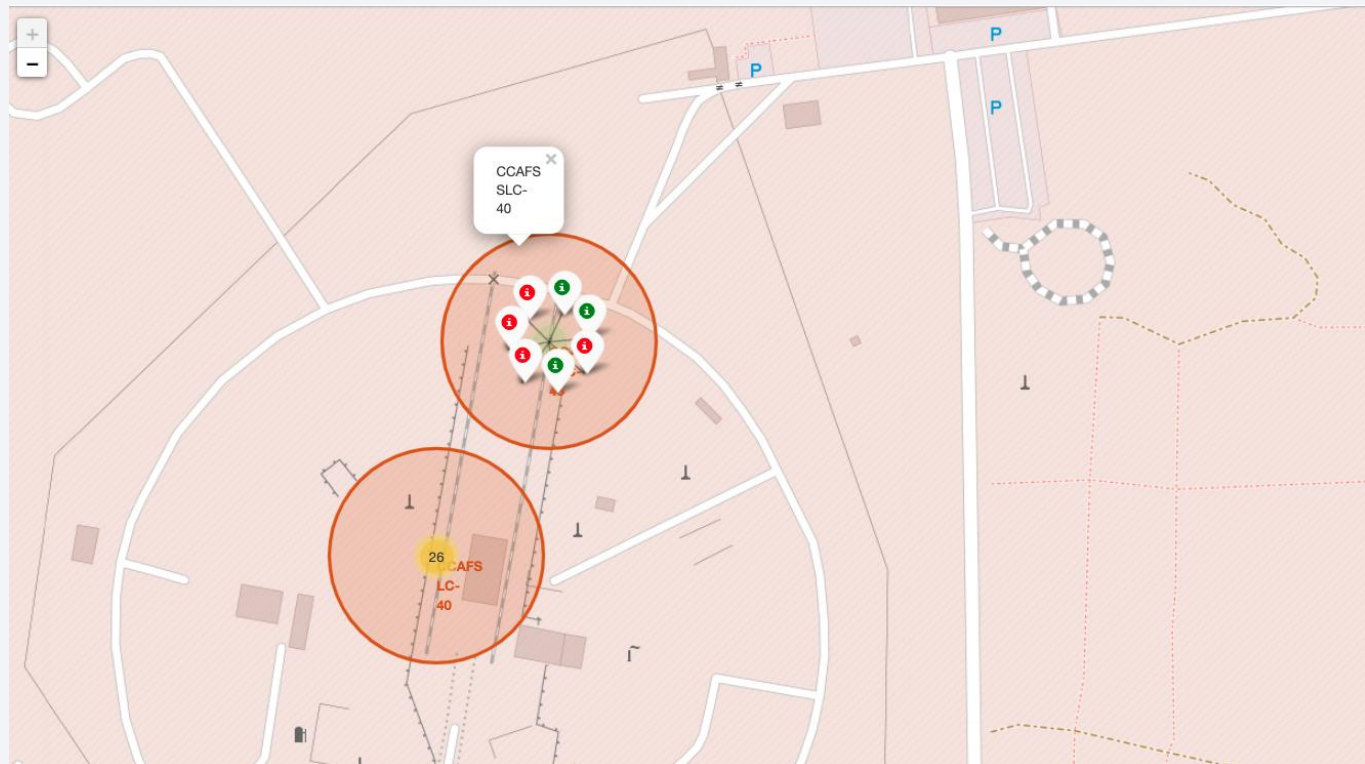
Build an Interactive Map with Folium

- In this notebook was marked all launch sites and added map objects like: markers, circles, lines for the purpose to mark the success/failure of launches for each site using Folium Map.
- The key purpose of creating Map was to find optimal location for building launch site

Link to the notebook here:



https://github.com/julia-khrabatyn/Data-Science-Project/blob/main/5_lab_jupyter_launch_site_location.ipynb



Build a Dashboard with Plotly Dash

- In this project was created an interactive Dashbard with Plotly Dash
- There were created:
 - a drop-down list for selecting launch site
 - a pie charts, that shows interactively % of total of successful/unsuccessful launches
 - slider of Payload Mass Range – for selecting interactively mass range
 - scatter chart showing Payload Mass vs success rate of booster version

Link to .py file with code here:



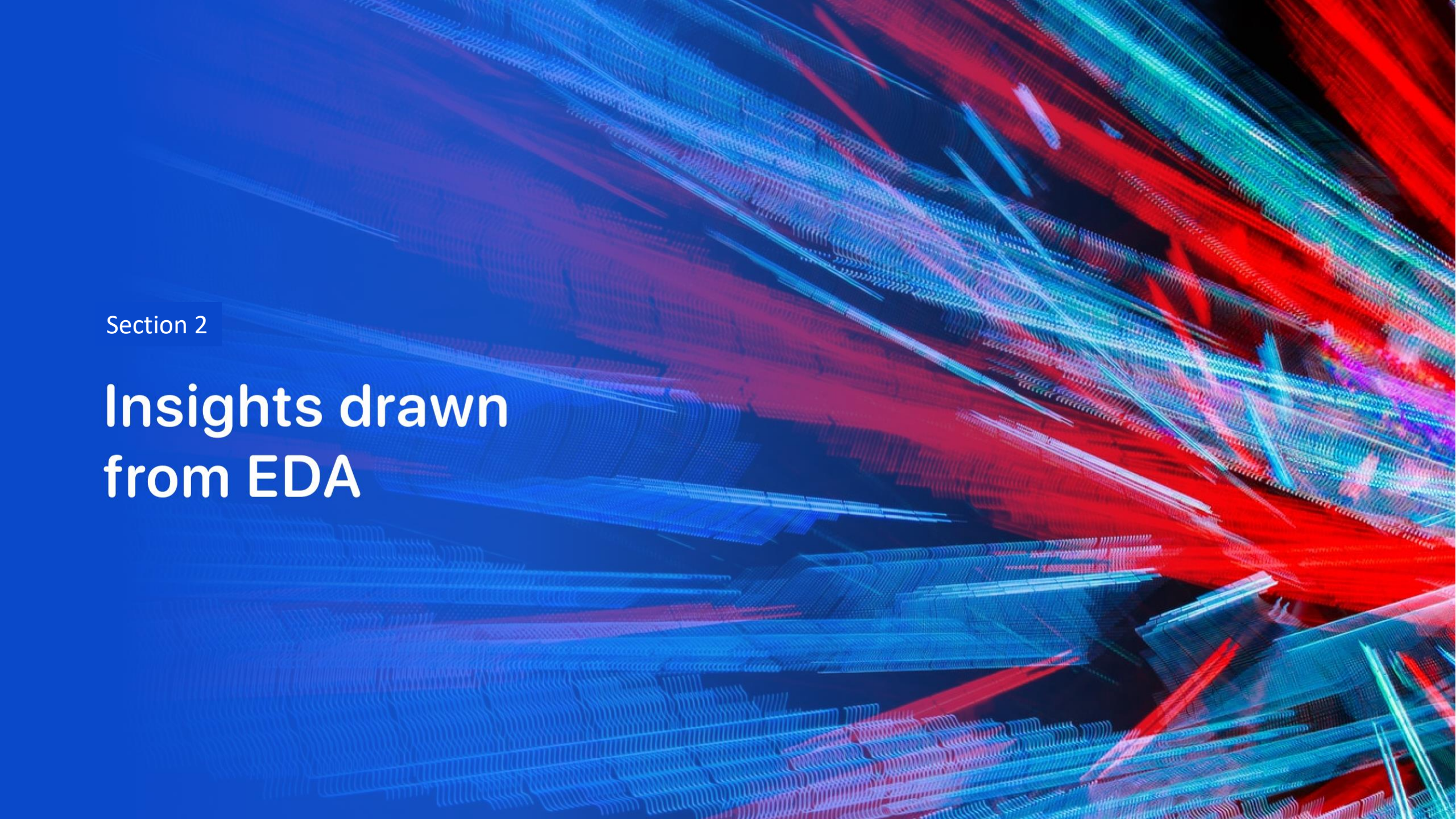
https://github.com/julia-khrabatyn/Data-Science-Project/blob/main/6_visual_analytics_plotly.py

Predictive Analysis (Classification)



Results

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

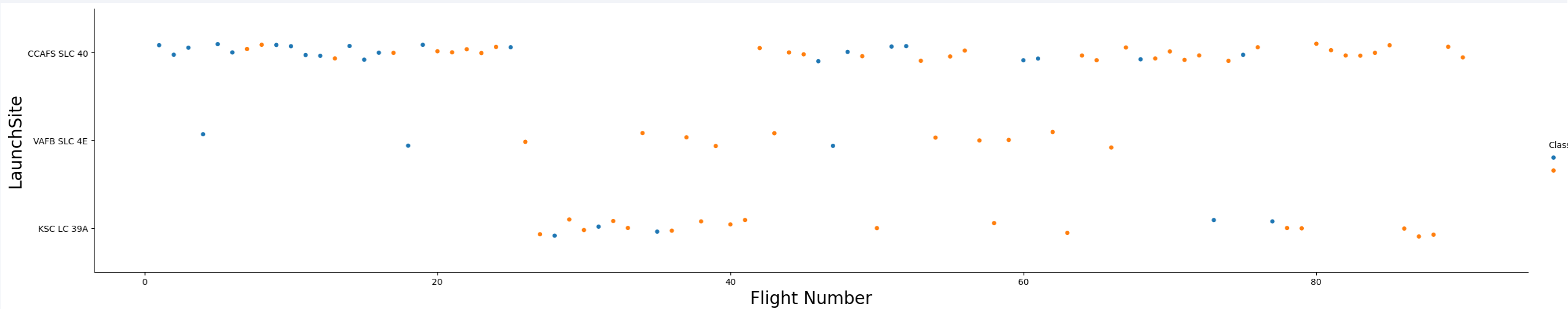
The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of red and cyan. A faint, light blue grid pattern is also visible, particularly in the lower-left quadrant. The overall effect is dynamic and technological.

Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

- Scatter plot of Flight Number vs. Launch Site



Explanations:

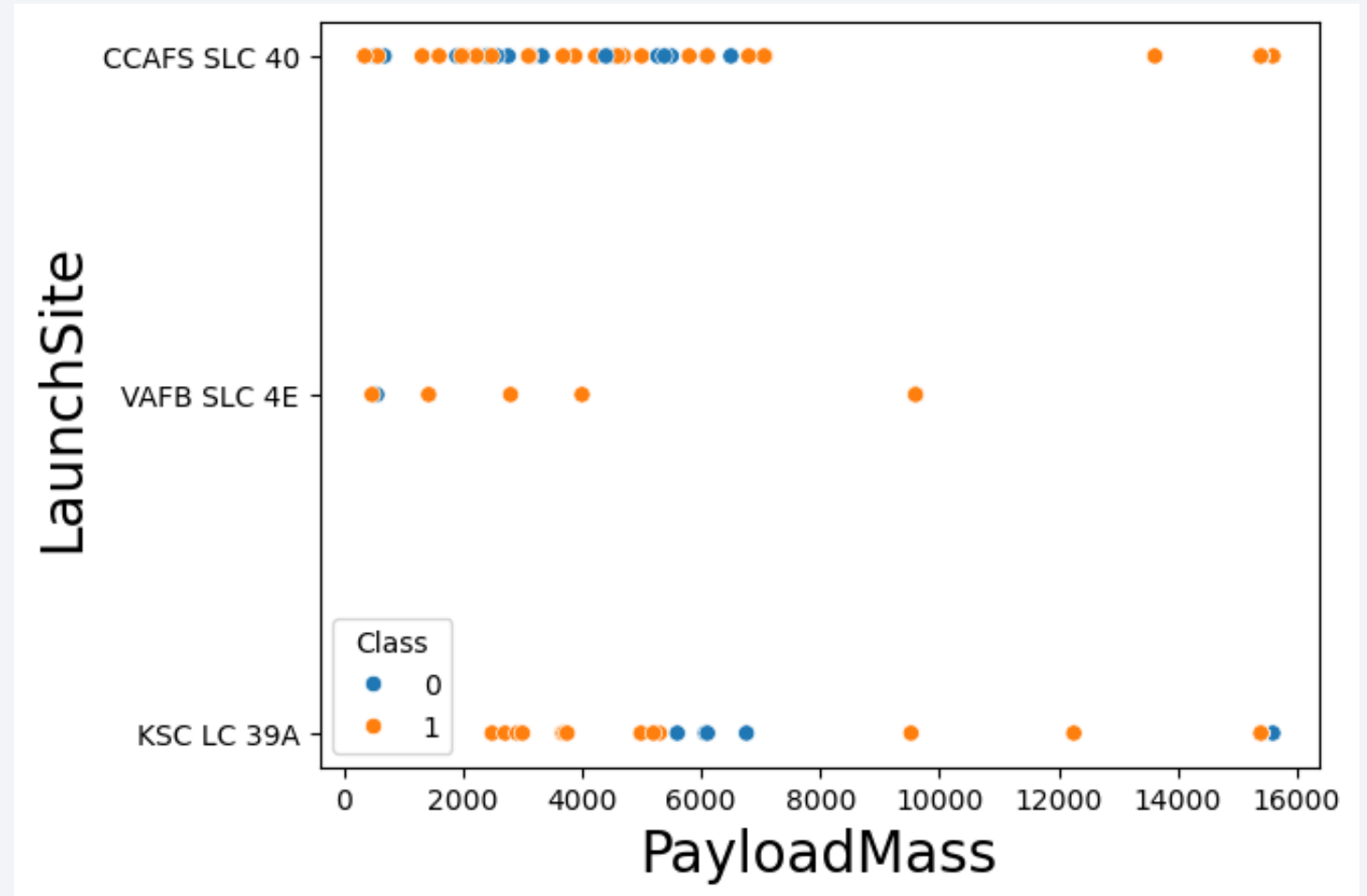
- ✓ Blue dots – show fail, while orange dots – show success
- ✓ VAFB SLC 4E and KSC LC 39A have higher success rate

Payload vs. Launch Site

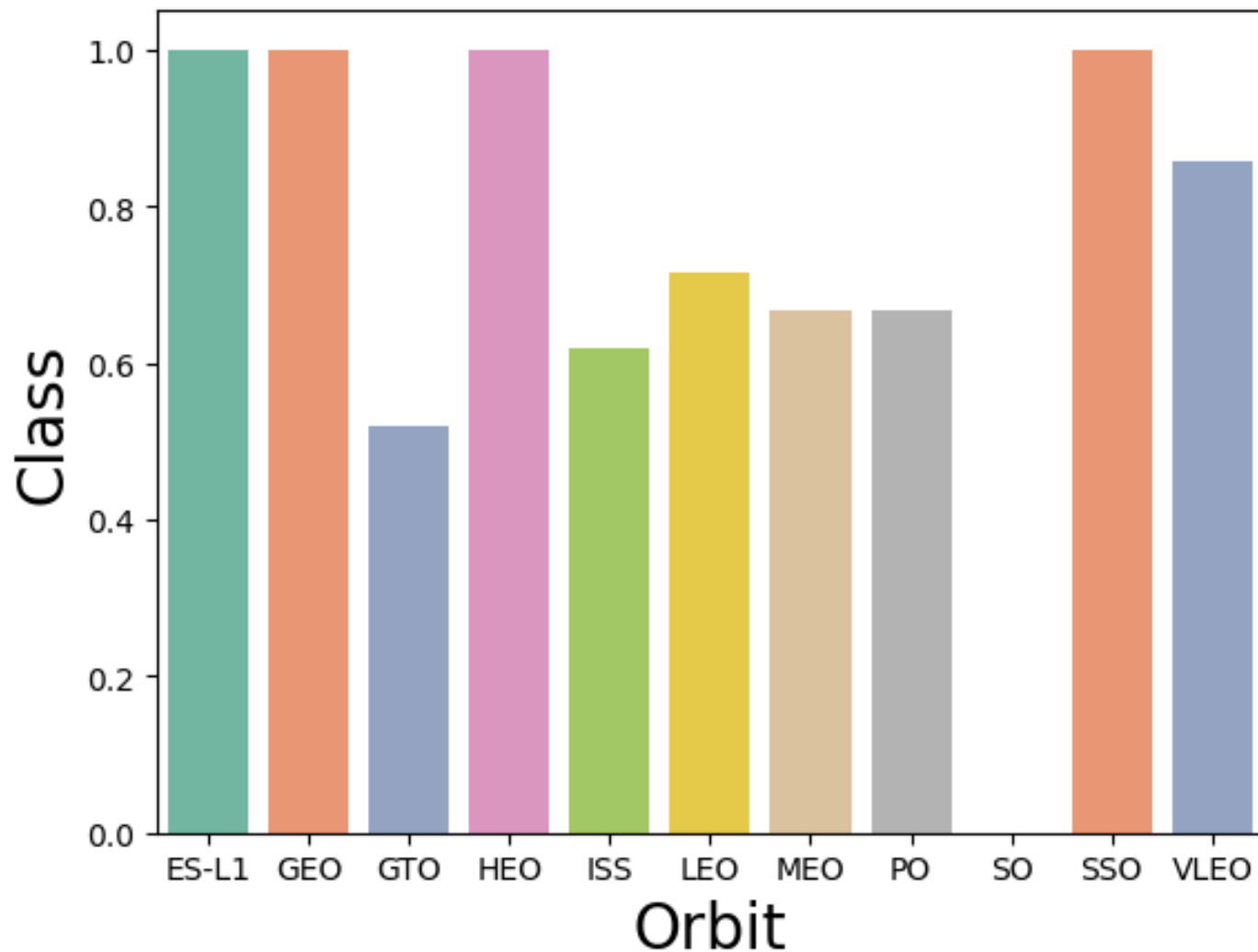
- Scatter plot of Payload vs. Launch Site

Explanations:

- ✓ In most cases the higher the payload mass – the higher success rate
- ✓ Most of the launches with a payload bigger than 7000 kg – was successful



Success Rate vs. Orbit Type



- Bar chart for the success rate of each orbit type

Explanations:

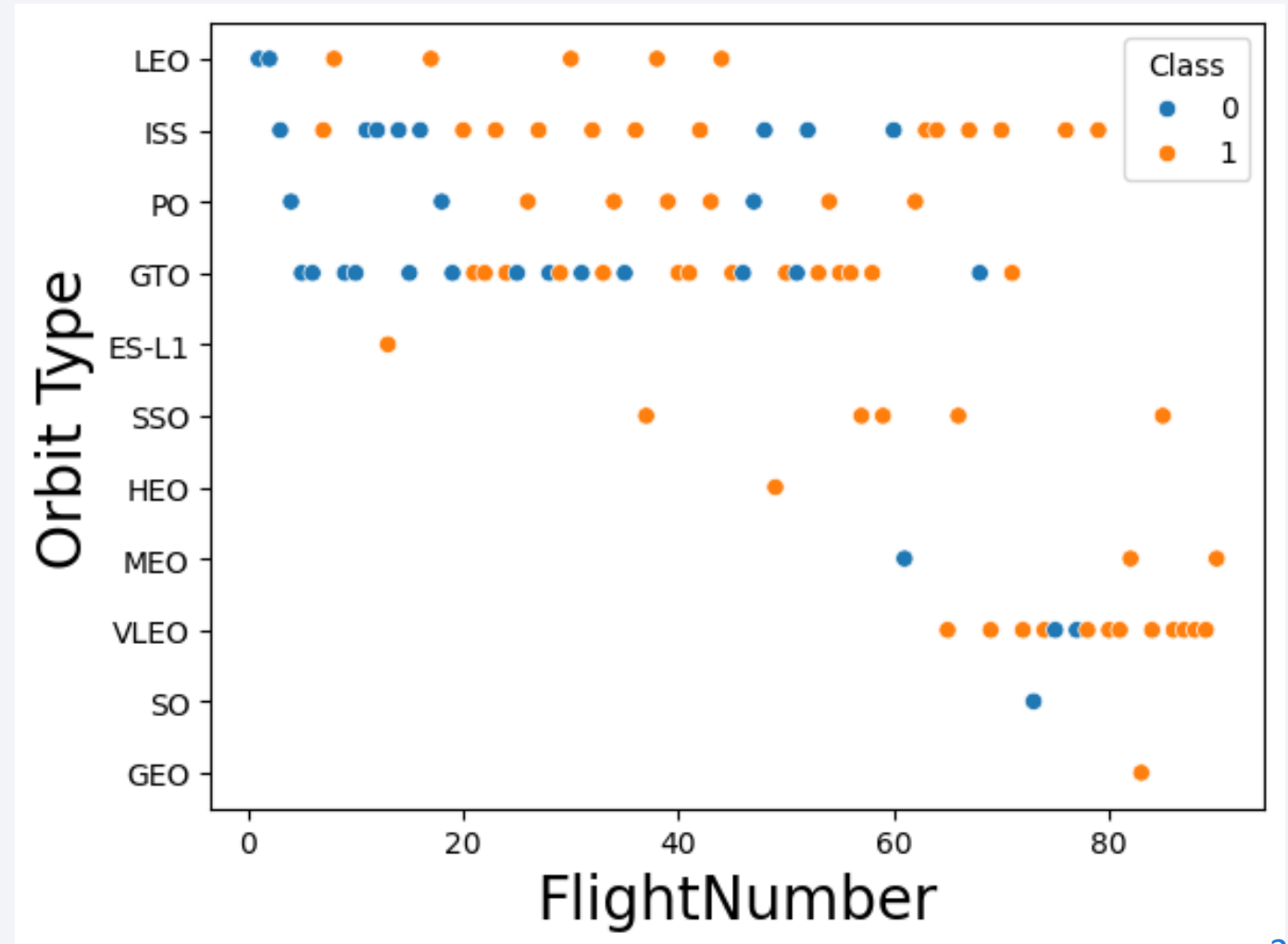
- ✓ 100% success rate: ES-L1, GEO, HEO, SSO
- ✓ 50% - 80% success rate: GTO, ISS, MEO, PO, LEO, VLEO
- ✓ 0% success rate: SO

Flight Number vs. Orbit Type

- Scatter point of Flight number vs. Orbit type

Explanations:

- ✓ We can observe that in the LEO orbit, success seems to be related to the number of flights. Conversely, in the GTO orbit, there appears to be no relationship between flight number and success.

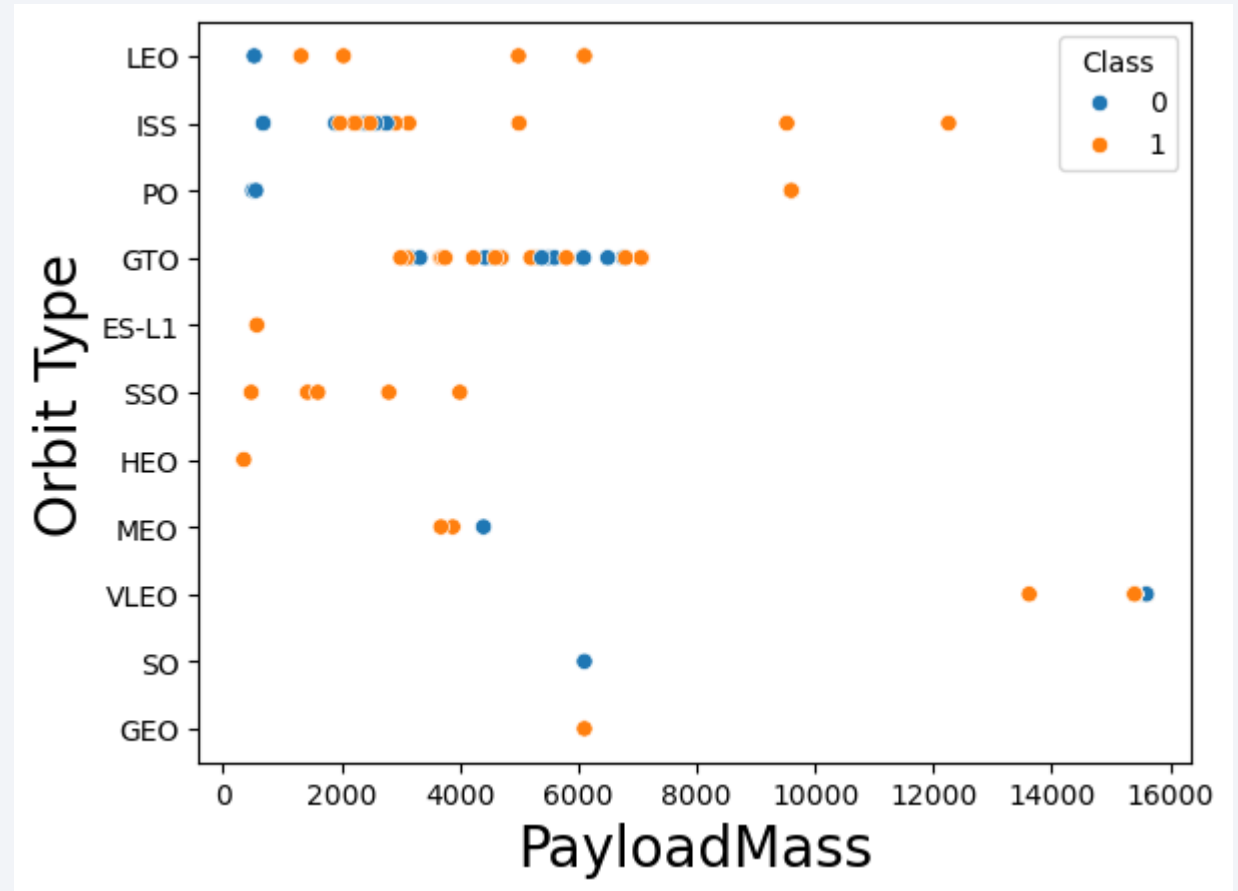


Payload vs. Orbit Type

- Scatter point of payload vs. orbit type

Explanations:

- ✓ With heavy payloads the successful landing or positive landing rate are more for PO, LEO and ISS.
- ✓ However, for GTO, it's difficult to distinguish between successful and unsuccessful landings as both outcomes are present.
- ✓ SSO have no failure at all in different mass range.

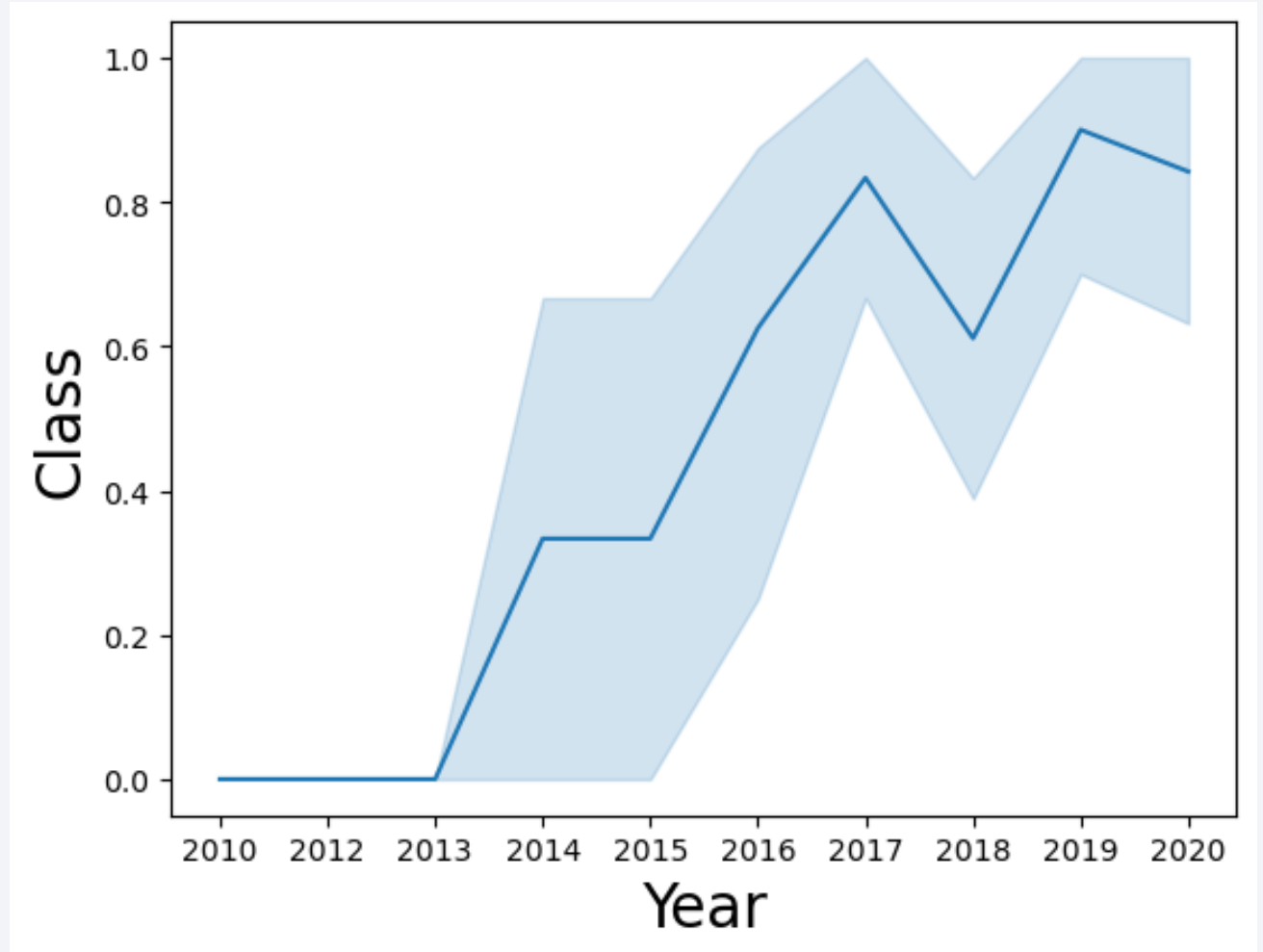


Launch Success Yearly Trend

- Line chart of yearly average success rate

Explanations:

- ✓ We can observe that the success rate since 2013 kept increasing till 2020.



All Launch Site Names

Launch Site names:

- CCAFS LC-40
- VAFB SLC-4E
- KSC LC-39A
- CCAFS SLC-40

Display the names of the unique launch sites in the space mission

```
In [49]: %sql SELECT DISTINCT Launch_Site FROM SPACEXTABLE
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[49]: Launch_Site
```

```
CCAFS LC-40
```

```
VAFB SLC-4E
```

```
KSC LC-39A
```

```
CCAFS SLC-40
```

The SELECT DISTINCT statement is used to return only unique values.

Launch Site Names Begin with 'CCA'

Here 5 records where launch sites begin with `CCA`

```
In [50]: %sql SELECT * FROM SPACEXTABLE WHERE Launch_Site LIKE 'CCA%' LIMIT 5

* sqlite:///my_data1.db
Done.
```

Out[50]:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

- LIMIT clause was used to select a limited number of records, in our case: 5.
- The LIKE operator is used in a WHERE clause to search for a specified pattern in a column.

Total Payload Mass

- Calculate the total payload carried by boosters from NASA

```
In [51]: %sql SELECT SUM(PAYLOAD_MASS__KG_) FROM SPACEXTABLE WHERE Customer = 'NASA (CRS)'
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[51]: SUM(PAYLOAD_MASS__KG_)  
         45596
```

The SUM() function returns the total sum of a numeric column. WHERE clause was used to filter data.

Average Payload Mass by F9 v1.1

- Calculate the average payload mass carried by booster version F9 v1.1

Display average payload mass carried by booster version F9 v1.1

```
In [52]: %sql SELECT AVG(PAYLOAD_MASS_KG_) FROM SPACE_TABLE WHERE Booster_Version = 'F9 v1.1'
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[52]: AVG(PAYLOAD_MASS_KG_)  
          2928.4
```

The AVG() function returns the average value of a numeric column. WHERE clause was used to filter data.

First Successful Ground Landing Date

- Here We Find the dates of the first successful landing outcome on ground pad

```
In [53]: %sql SELECT MIN(Date) FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (ground pad)'
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[53]: MIN(Date)  
2015-12-22
```

The MIN() function returns the smallest value of the selected column. WHERE clause was used to filter data.

Successful Drone Ship Landing with Payload between 4000 and 6000

- List the names of boosters which have successfully landed on drone ship and had payload mass greater than 4000 but less than 6000 **by using BETWEEN clause for filtering our data**

```
In [54]: %sql SELECT * FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (drone ship)' AND PAYLOAD_MASS_KG_ BETWEEN 4000 AND 6000
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[54]:
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
------	------------	-----------------	-------------	---------	------------------	-------	----------	-----------------	-----------------

2016-05-06	5:21:00	F9 FT B1022	CCAFS LC-40	JCSAT-14	4696	GTO	SKY Perfect JSAT Group	Success	Success (drone ship)
2016-08-14	5:26:00	F9 FT B1026	CCAFS LC-40	JCSAT-16	4600	GTO	SKY Perfect JSAT Group	Success	Success (drone ship)
2017-03-30	22:27:00	F9 FT B1021.2	KSC LC-39A	SES-10	5300	GTO	SES	Success	Success (drone ship)
2017-10-11	22:53:00	F9 FT B1031.2	KSC LC-39A	SES-11 / EchoStar 105	5200	GTO	SES EchoStar	Success	Success (drone ship)

The BETWEEN operator selects values within a given range. WHERE clause was used to filter data.

Total Number of Successful and Failure Mission Outcomes

- Calculate the total number of successful and failure mission outcomes

As we can see we get 100 times of success and 1 failure mission

```
In [55]: %sql SELECT Mission_Outcome, COUNT(*) AS Total_Num FROM SPACEXTABLE GROUP BY Mission_Outcome
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
Out[55]:
```

Mission_Outcome	Total_Num
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

The COUNT() function returns the number of rows that matches a specified criterion. The GROUP BY statement groups rows that have the same values into summary rows

Boosters Carried Maximum Payload

- List the names of the booster which have carried the maximum payload mass

```
In [56]: %sql SELECT Booster_Version FROM SPACEXTABLE WHERE PAYLOAD_MASS_KG_ = (SELECT MAX(PAYLOAD_MASS_KG_) FROM SPACEXTABLE)
* sqlite:///my_data1.db
Done.
Out[56]: Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7
```

Here in our sql query we use a subquery in the WHERE clause!

2015 Launch Records

- List the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015
- SQL query here: %sql SELECT substr(Date, 6,2) AS month, DATE, Booster_Version, Launch_Site, [Landing_Outcome] FROM SPACEXTABLE WHERE [Landing_Outcome] = 'Failure (drone ship)' AND substr(Date,0,5)='2015'

```
In [57]: %sql SELECT substr(Date, 6,2) AS month, DATE, Booster_Version, Launch_Site, [Landing_Outcome] FROM SPACEXTABLE WHERE  
* sqlite:///my_data1.db  
Done.
```

```
Out[57]:
```

	month	Date	Booster_Version	Launch_Site	Landing_Outcome
	01	2015-01-10	F9 v1.1 B1012	CCAFS LC-40	Failure (drone ship)
	04	2015-04-14	F9 v1.1 B1015	CCAFS LC-40	Failure (drone ship)

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order
- SQL query here: %sql SELECT [Landing_Outcome], COUNT(*) AS Num_of_outcomes FROM SPACEXTABLE WHERE DATE BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY [Landing_Outcome] ORDER BY Num_of_outcomes DESC
- Here we used a combination of WHERE clause and conditions BETWEEN, AND for filtering data

```
In [59]: %sql SELECT [Landing_Outcome], COUNT(*) AS Num_of_outcomes FROM SPACEXTABLE WHERE DATE BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY [Landing_Outcome] ORDER BY Num_of_outcomes DESC
```

* sqlite:///my_data1.db
Done.

Out[59]:

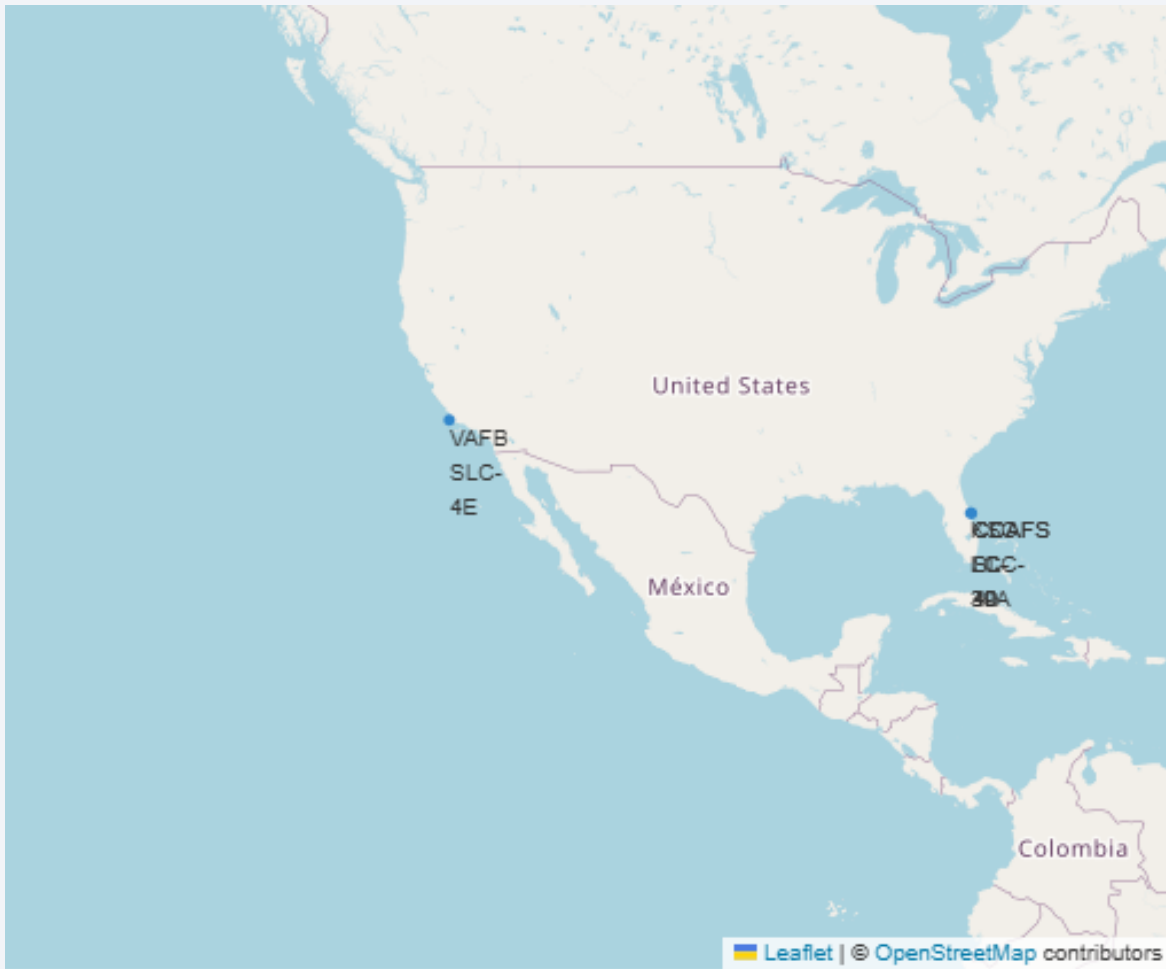
Landing_Outcome	Num_of_outcomes
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

A satellite view of Earth from space, showing the curvature of the planet and the glowing city lights of the Eastern United States and parts of Canada at night. The background is a deep blue gradient.

Section 3

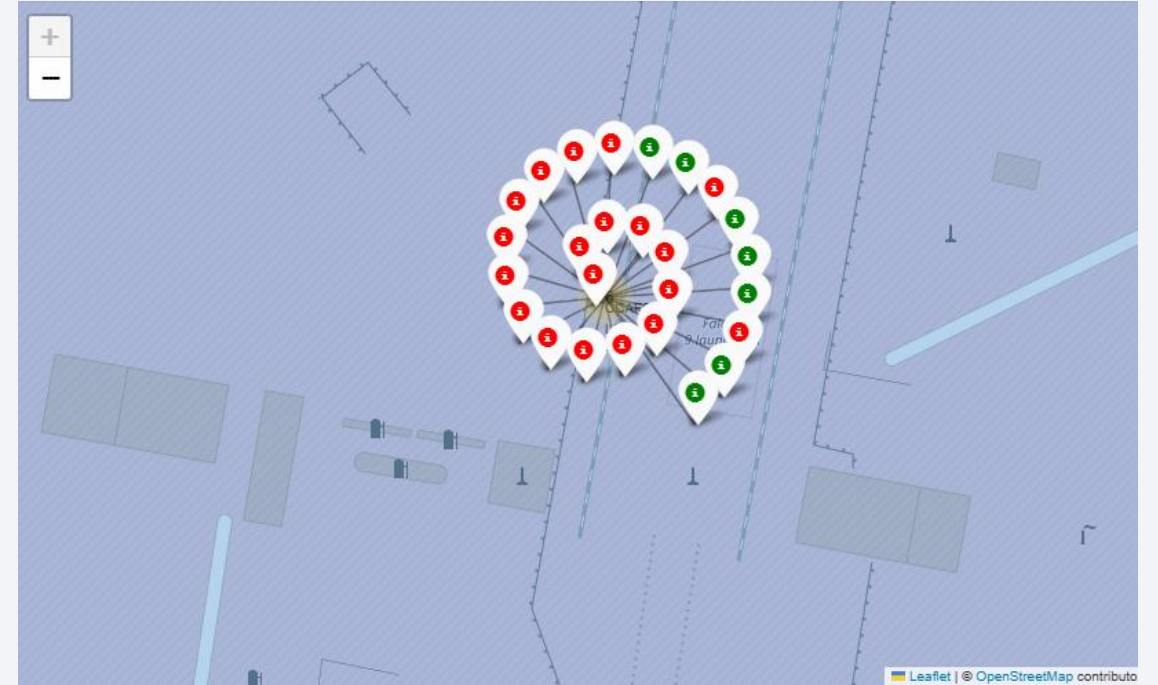
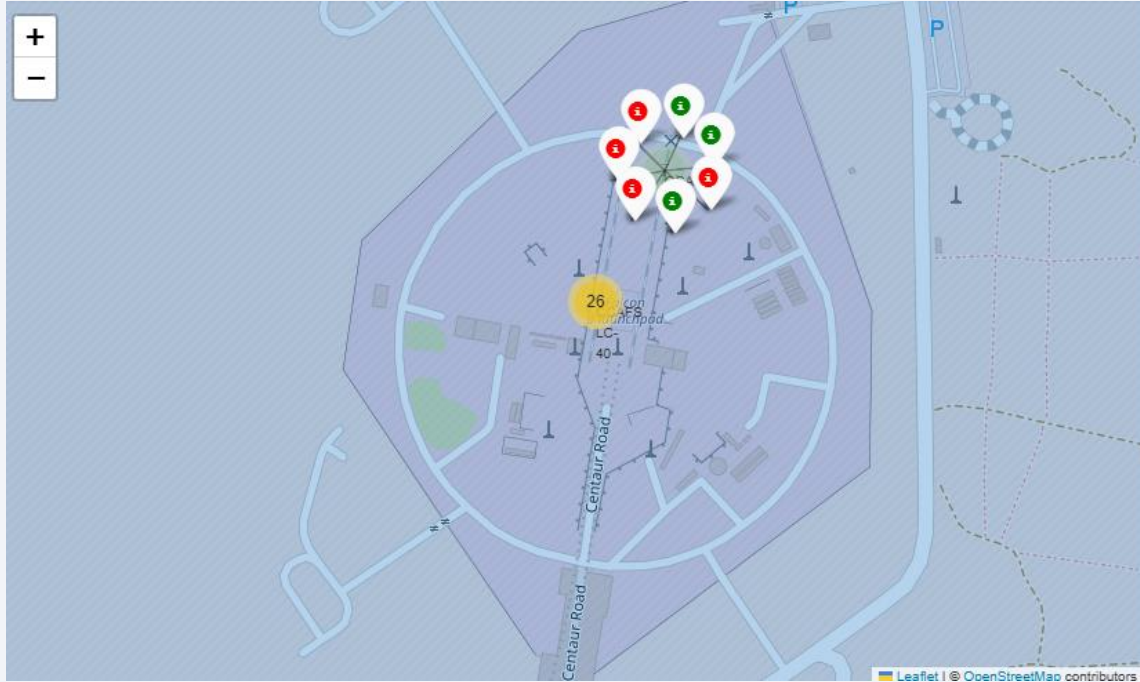
Launch Sites Proximities Analysis

Success/Failed Launch Sites on a Map



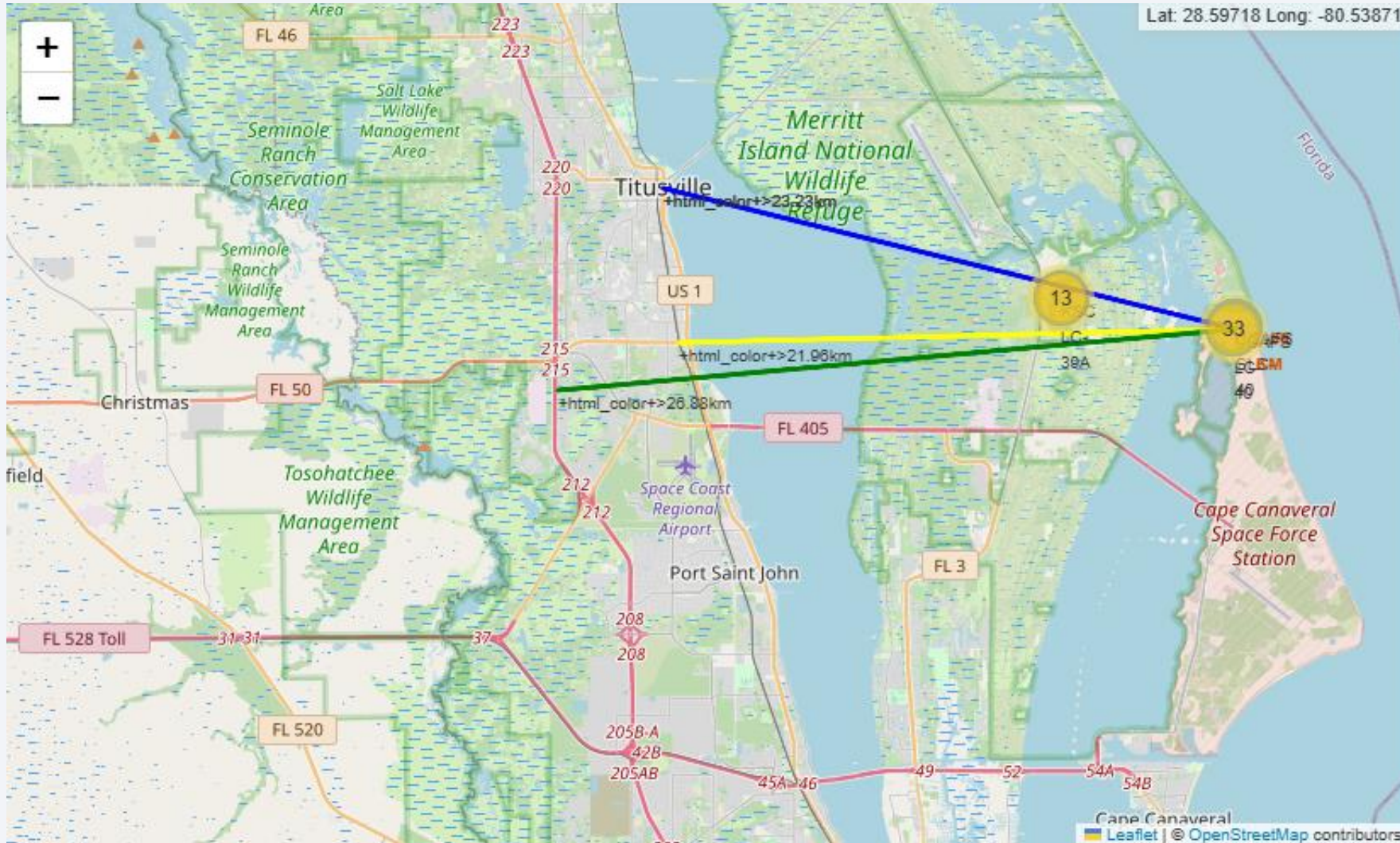
The generated map with marked launch sites was performed by creating and adding folium.Circle and folium.Marker for each launch site on the site map

Launch outcomes for CCAFS SLC-40



- ✓ Green markers show successful launch
- ✓ Red markers show failed launch

The distances between a launch site to its proximities



- ✓ Blue line on the Map follow to nearest city
- ✓ Yellow line on the Map follow to nearest railway
- ✓ Green line on the Map follow to nearest highway

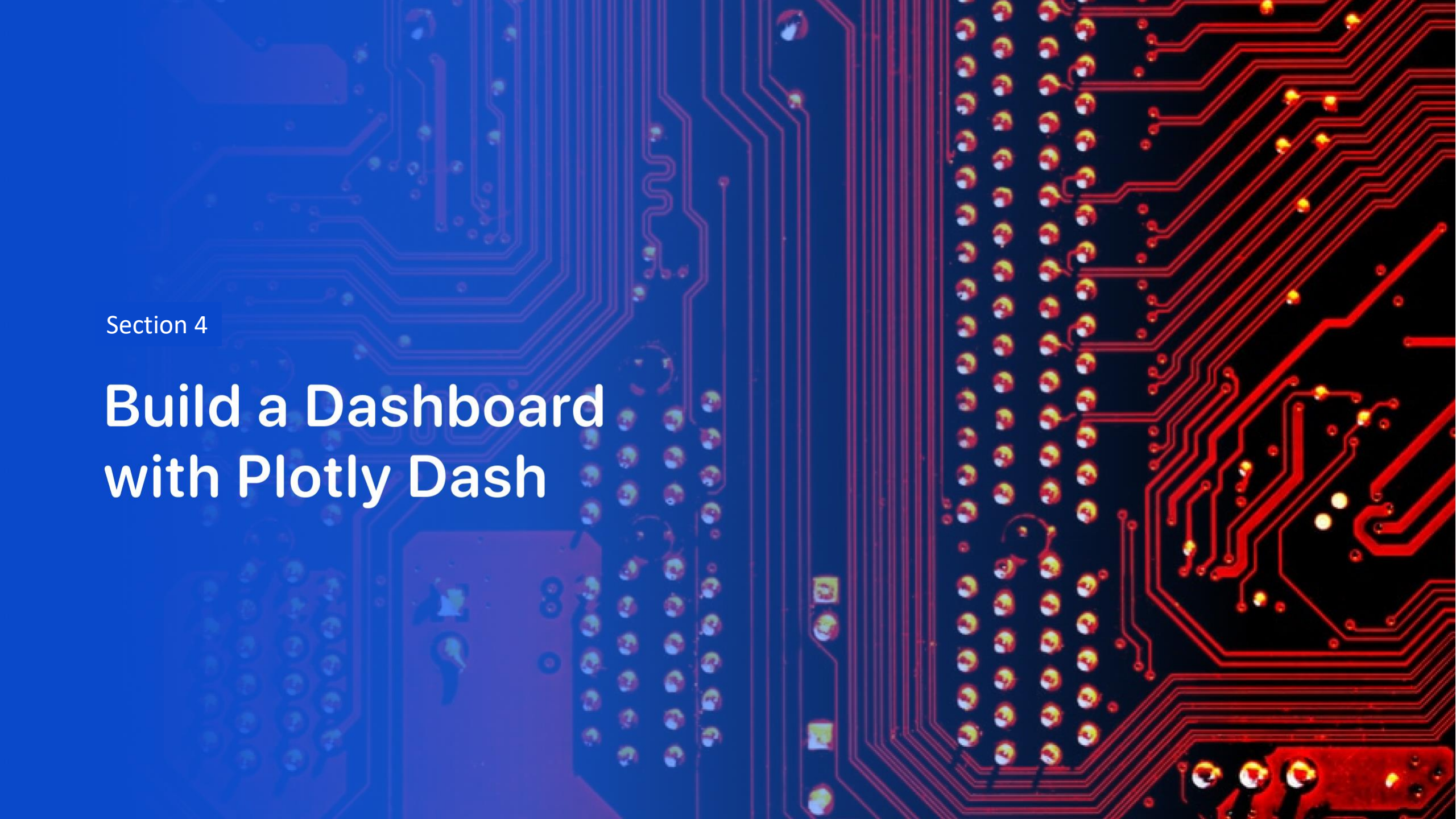
Calculations for CCAFS SLC-40

City distance: 23.235 km

Railway distance: 21.961 km

Highway distance: 26.880 km

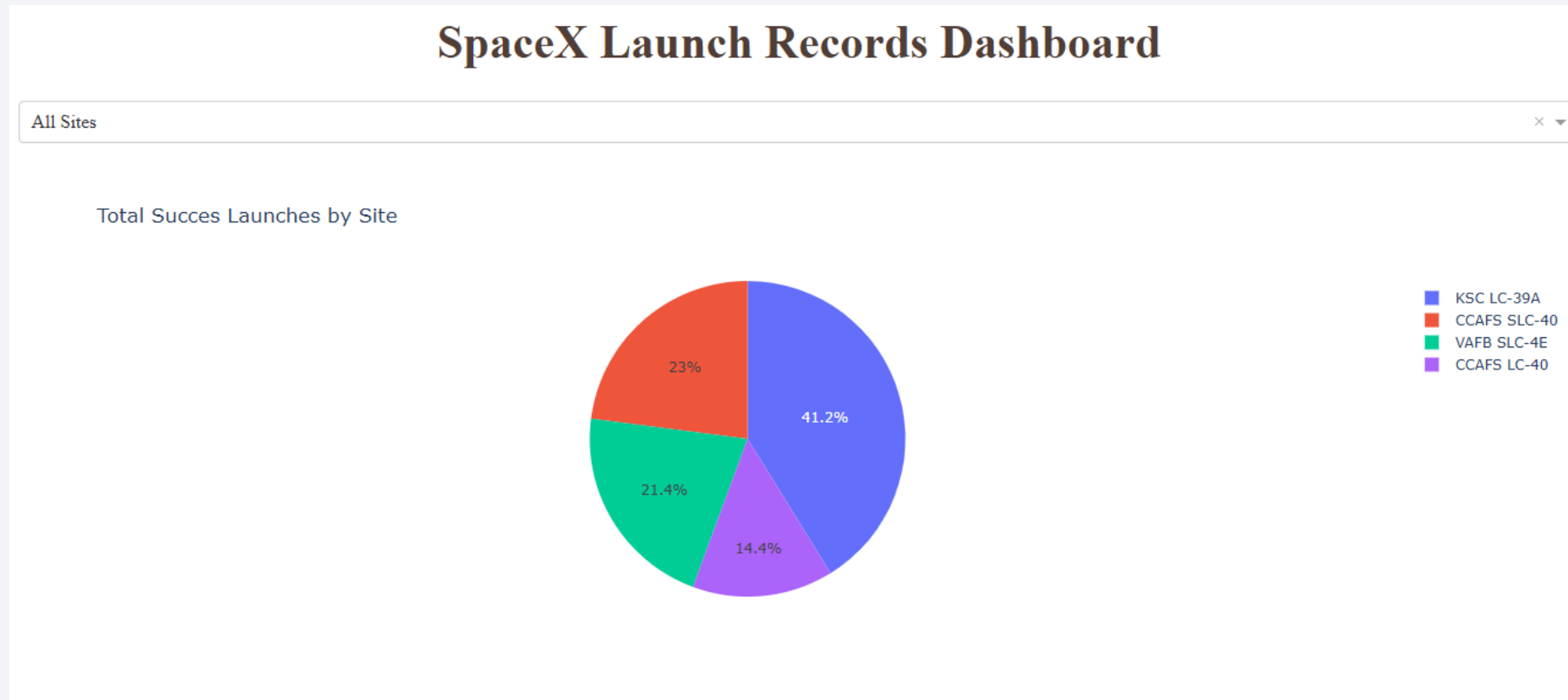
Coastline distance: 0.863 km



Section 4

Build a Dashboard with Plotly Dash

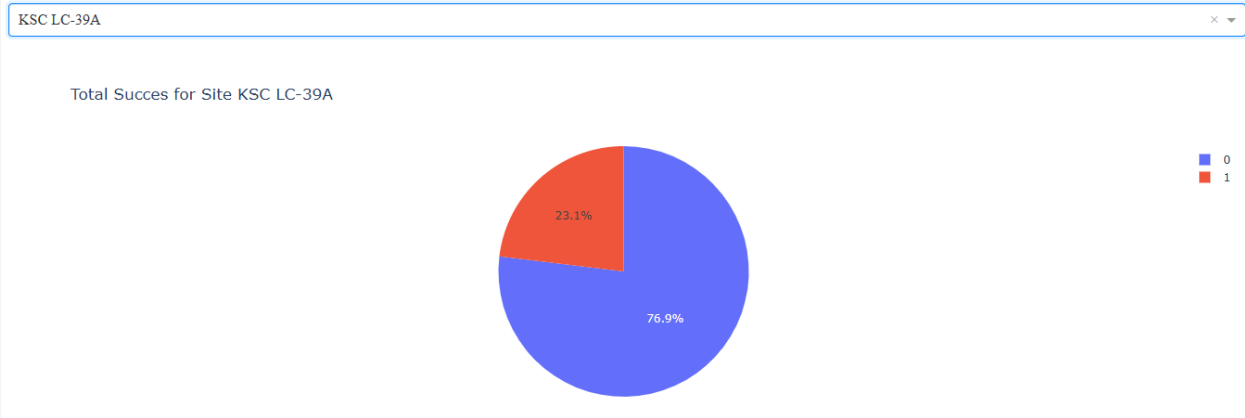
Total Success Launches by Site



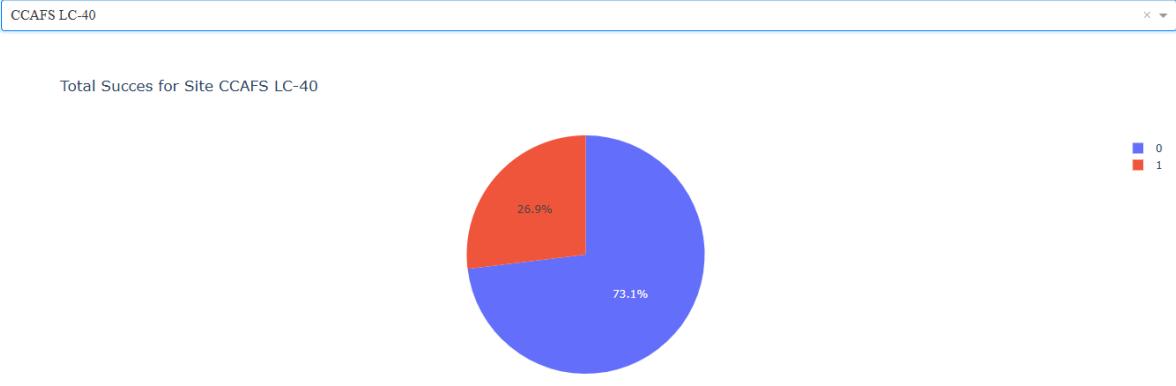
KSC LC-39A is the most successful launch in in comparison with other launches (41,2 %)

Total Success for each Launch in descending order

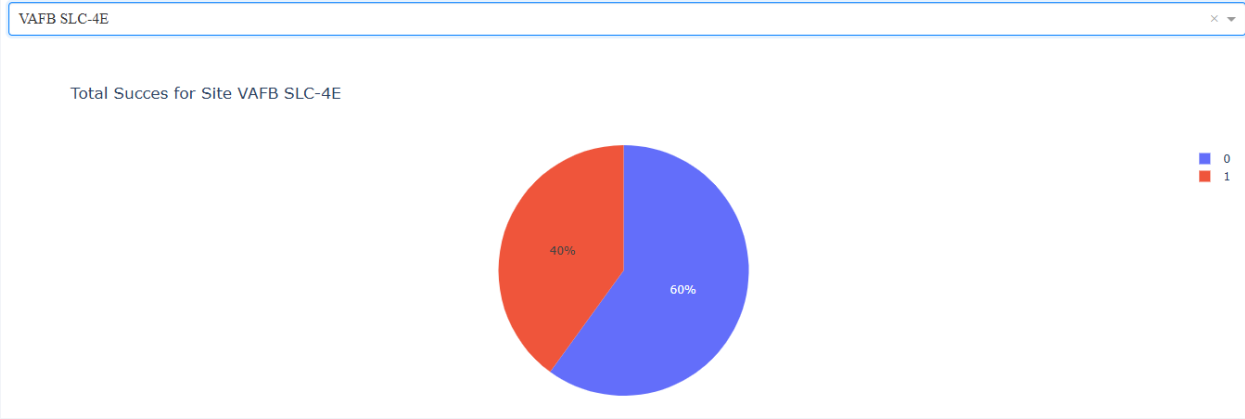
SpaceX Launch Records Dashboard



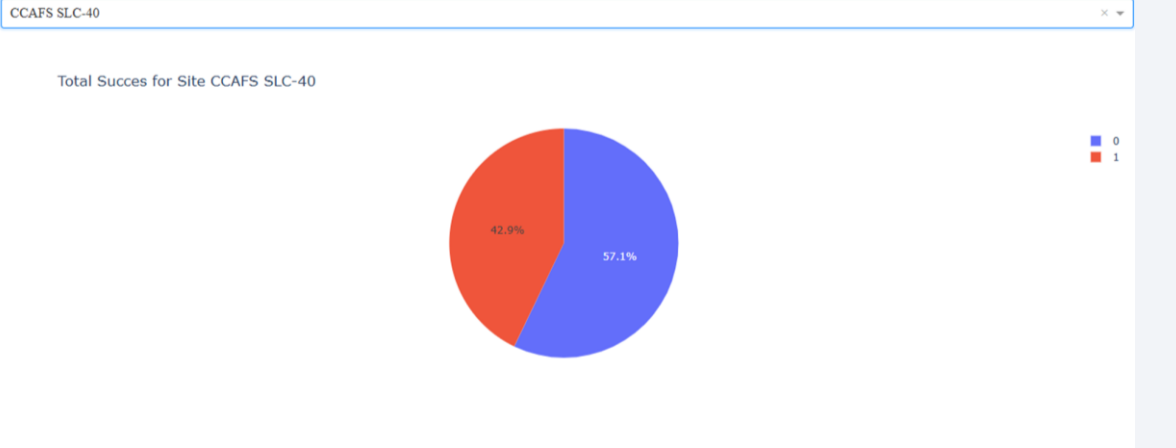
SpaceX Launch Records Dashboard



SpaceX Launch Records Dashboard



SpaceX Launch Records Dashboard



Payload Mass and Success of All Sites



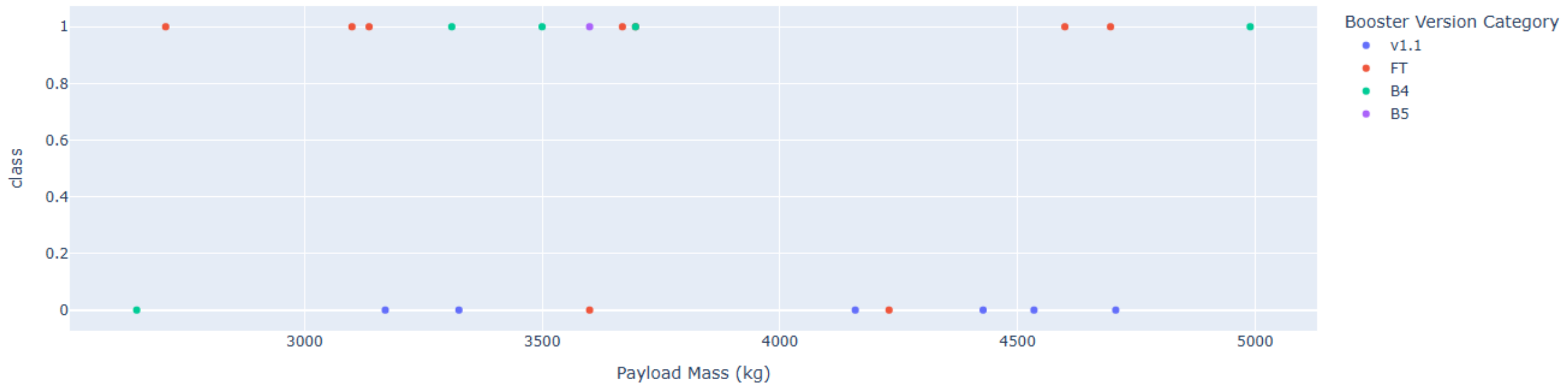
Class 0 - shows fail, while class 1 – shows success!

Payload Mass and Success of All Sites

Payload range (Kg):



Correlation for Payload Mass (kg) and Success of All Sites

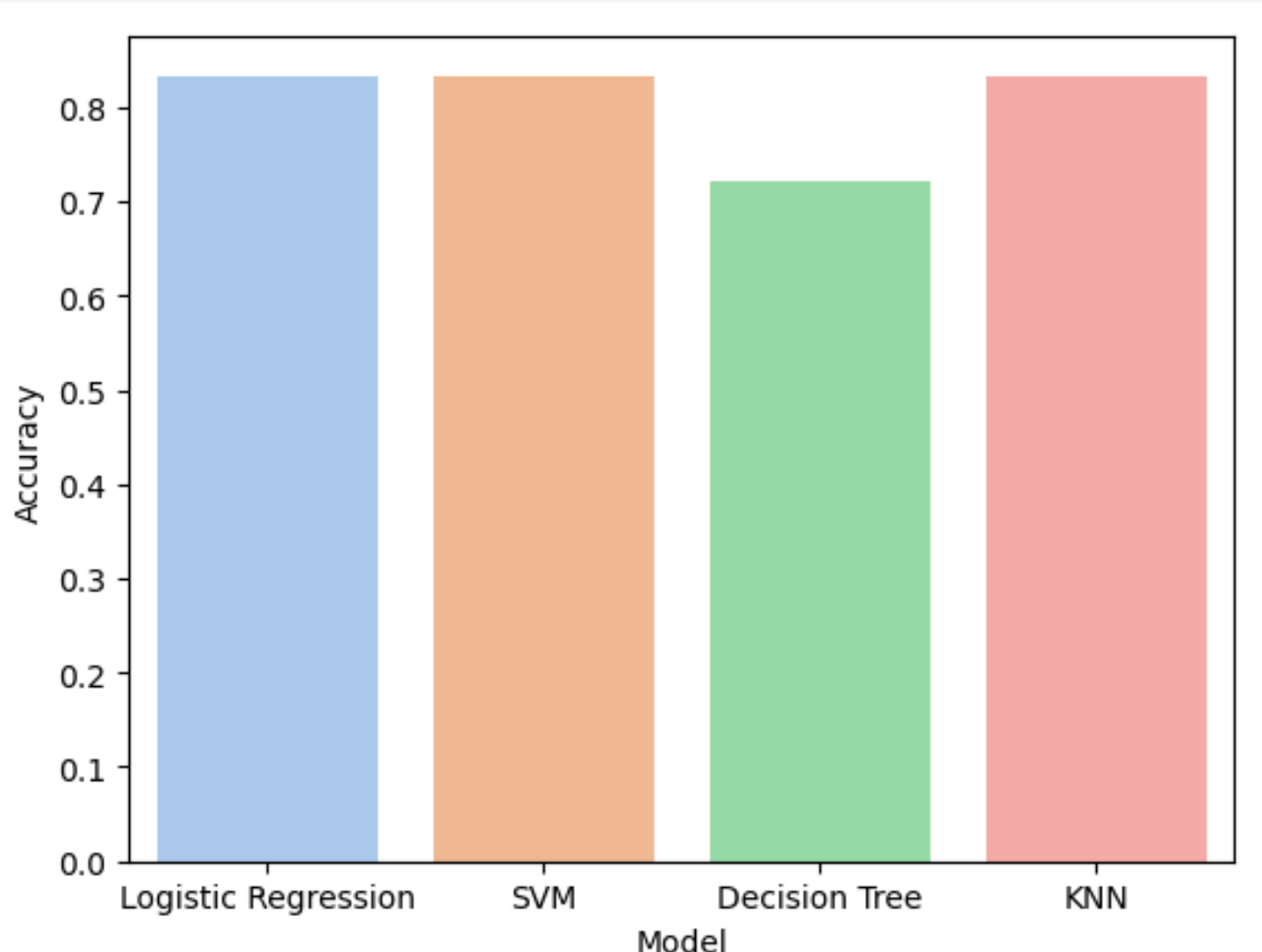


Payload mass in range 2500 – 5000 give the highest success rate!

Section 5

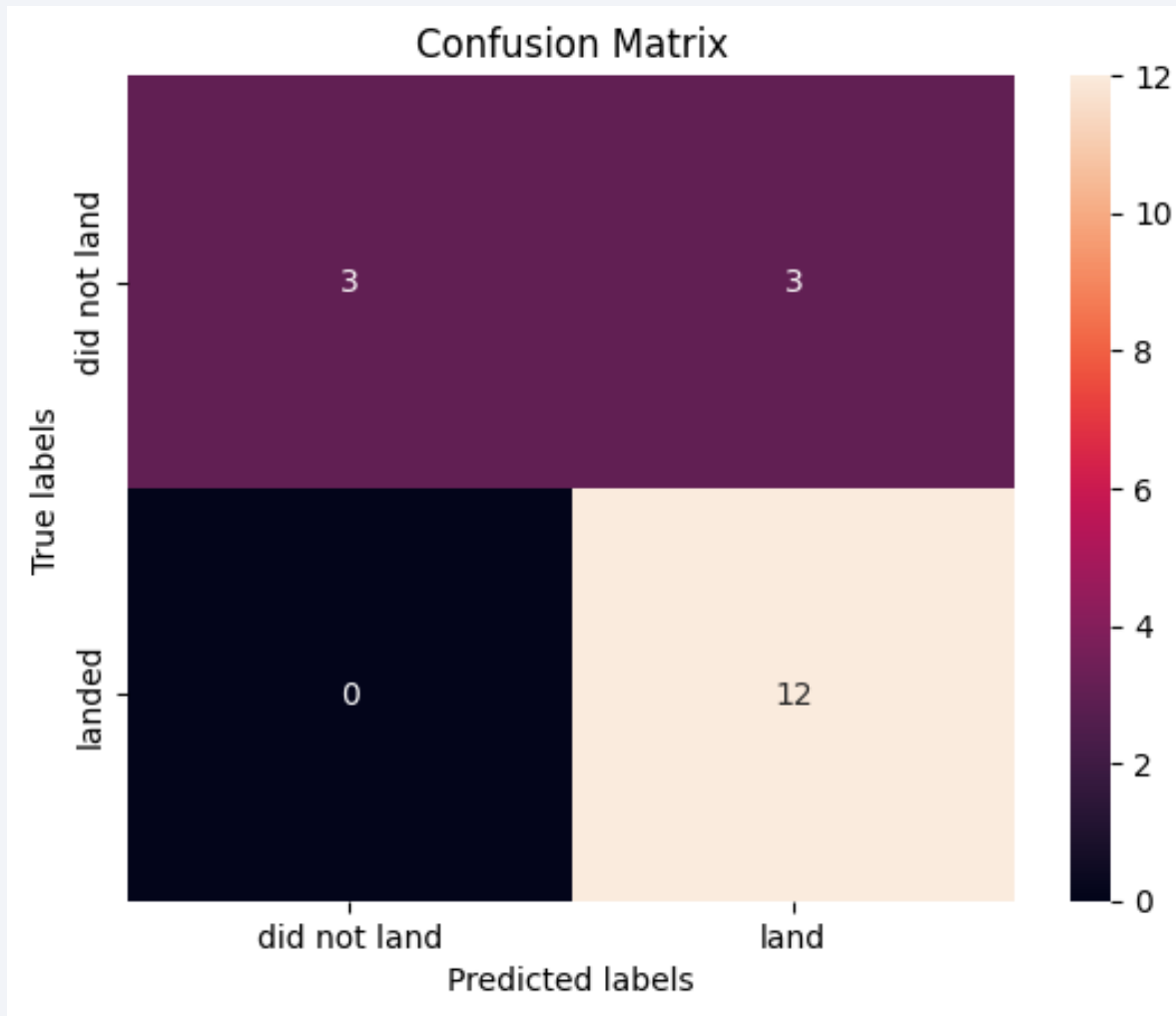
Predictive Analysis (Classification)

Classification Accuracy



- All of the models perform quite well, except the tree model which give 72,2% of accuracy on test data, however Decision Tree has the highest accuracy on train data = 88,75%

Confusion Matrix



- As we can see the problem with this model is availability of 3 False Positives

Confusion Matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Conclusions

- Logistic Regression, SVM, KNN – are good models by accuracy for our dataset .
- Higher weighted payloads perform better (payload mass in range 2500 – 5000 give the highest success rate!)
- the success rate since 2013 kept increasing till 2020.
- KSC LC-39A had the most successful launch in in comparison with other sites (41,2 %)
- ES-L1, GeO, HEO, SSO Orbits had the best rate in comparison with other
- Most of the launch sites were close to the equator and all of them were close to the coast

Thank you!

